

Do CLP ao Celular

Modbus TCP, MQTT e um sistema completo em 5 horas

Gabriel de Oliveira e Silva

Monitoria de Introdução às Redes de Comunicação, UFPE

2026.2

MQTT

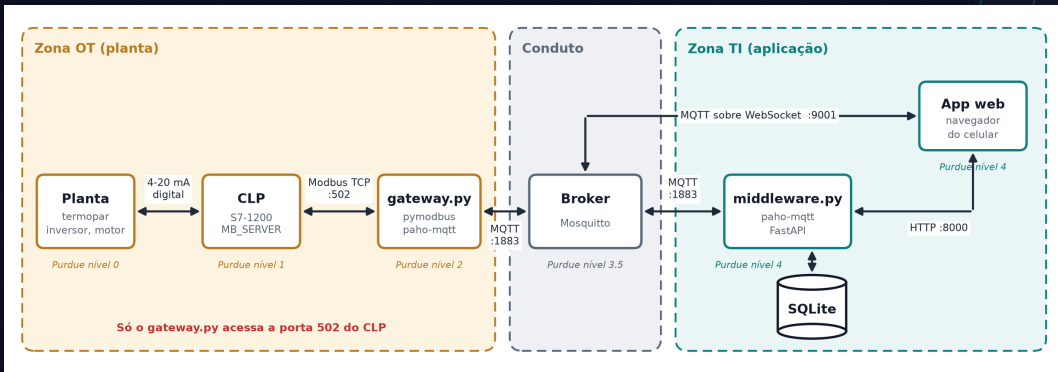
O que cada grupo constrói hoje

- O **CLP** mede a temperatura e aciona o motor pelo inversor
- Um **gateway** em Python lê o CLP por **Modbus TCP** e publica num **broker MQTT**
- Um **middleware** em Python grava tudo num banco e serve o **app**
- Pelo **celular**, o operador comanda o motor

Cada grupo é uma bancada

bancada1, bancada2, ... Cada uma com o seu CLP e o seu **broker MQTT**.

A arquitetura



Cronograma

Horário	Etapa	Ao final você vê
0:00	Preparação	ambiente sem erro
0:15	CLP: <i>FB_Planta</i>	temperatura na Watch Table
0:50	1. Ler o CLP	temperatura no terminal
1:20	2. Broker	recusa quem não tem senha
2:10	3. Publicar	tópicos no MQTT Explorer
2:45	4. Comandar	motor girando por MQTT
3:30	5. Banco e app	bancada no celular
4:10	6. Segurança	watchdog, ACL, Wireshark
4:40	Fechamento	CSV entregue

Intervalos às 2:00 e às 4:05.

Regras da bancada

Antes de qualquer comando

- O motor só liga com o monitor olhando a bancada
- Testes de parada (watchdog e emergência): só o monitor
- Nada de mexer na fiação com a bancada energizada
- Ataques: só nesta rede de laboratório

Preparação: o ambiente

Terminal dentro da pasta *projeto_aluno*. Sem erro, está pronto:

```
.venv\Scripts\python -c "import pymodbus, paho.mqtt, fastapi"
```

Criar o arquivo de configuração:

```
copy config.exemplo.env config.env
```

Por que um arquivo só

IPs, portas e senhas ficam num único lugar, fora do git (ADR-025).

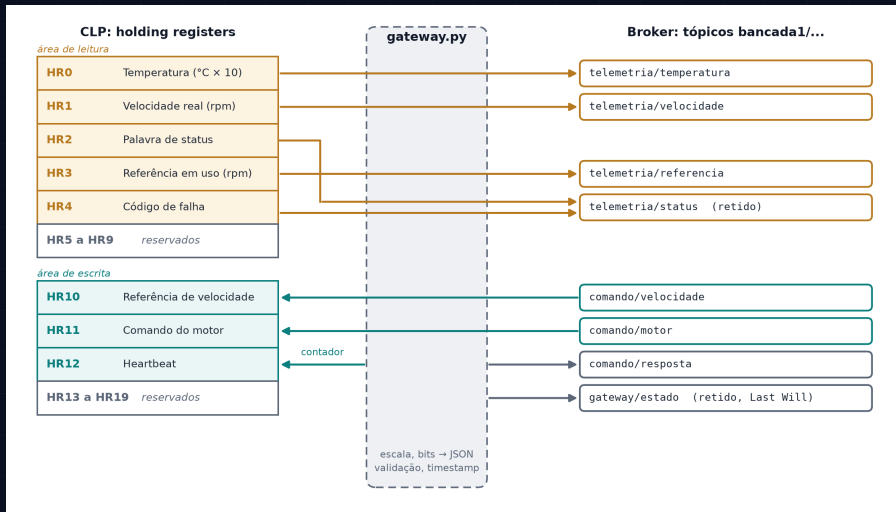
Preparação: o config.env

Chave	O que colocar
<i>BANCADA</i>	<i>bancadaN</i> , o número do grupo
<i>MODBUS_HOST</i>	IP do seu CLP: <i>192.168.0.(10+N)</i>
<i>MQTT_HOST</i>	<i>127.0.0.1</i> : o seu broker (etapa 2)
senhas do MQTT	vocês criam na etapa 2

Serviço do Mosquitto parado: *Get-Service mosquitto*

Teste: *ping 192.168.0.1N*

CLP: o contrato de dados



CLP: o mapa de registradores

Registrador	Conteúdo	Escreve
HR0	temperatura, °C × 10	CLP
HR1	velocidade (rpm)	CLP
HR2	status, um bit por informação	CLP
HR3	referência aplicada (rpm)	CLP
HR4	código de falha	CLP
HR10	referência pedida (rpm)	gateway
HR11	0 parar, 1 ligar, 2 reconhecer	gateway
HR12	heartbeat (contador)	gateway

CLP: duas ideias do contrato

Pedido $\neq$ aplicado

HR10 é o que foi **pedido**.

HR3 é o que o CLP **aceitou**.

Pedi 5000 rpm? O CLP limita a 1700 e o app mostra a diferença.

Heartbeat

O gateway muda HR12 a cada segundo.

Parou de mudar por 3 s? O CLP para o motor sozinho.

A segurança fica no CLP.

CLP: carregar o FB_Planta

Passo a passo em *clp/LEIAME.md*:

1. IP da bancada: $192.168.0.(10+N)$
2. Tabela de variáveis *Planta*
3. Importar *FB_Planta.scl* (External source files)
4. Chamar o FB no *Main [OB1]*, depois do *MB_SERVER*
5. Compilar, carregar, RUN

Checkpoint

Watch Table: segure o termopar e veja *holdingRegister[0]* subir.

Plano B: o CLP simulado

Se o CLP não compilar ou não responder, no *config.env*:

```
MODO=simulado MODBUS_HOST=127.0.0.1 MODBUS_PORT=5020
```

e num terminal separado:

```
.venv\Scripts\python clp_simulado.py
```

Mesmo mapa, mesma lógica. Teclas: **l** local, **r** remoto, **e** emergência.

Etapa 1: ler o CLP em Python

Em *gateway/gateway.py*, função *ler_clp*: FC 3 em HR0 a HR9.

1. *self.clp.read_holding_registers(c.LEITURA_INICIO, count=...)*
2. Exceção (cabo, CLP fora): *log.warning* e *None*
3. *resposta.isError()*: exceção Modbus, *None*
4. Deu certo: *resposta.registers*

A conversão já existe: *c.decodificar_leitura* em *contrato.py*.

Etapa 1: checkpoint

```
.venv\Scripts\python gateway\gateway.py
```

A cada segundo:

```
T=24.8 °C  v=0 rpm  ref=0 rpm  remoto=True  falha=0
```

Para pensar

- Como 65336 vira -200? (*para_int16*)
- Por que ler 10 registradores numa requisição só?

Etapa 2: levantar o broker

O que é

O carteiro do MQTT: recebe cada publicação e entrega a quem assinou o tópico.

Na arquitetura, o **conduto** entre OT e TI.

Dois arquivos, escritos por vocês

broker/mosquitto.conf: como o broker funciona

broker/acl: quem pode o quê

Um broker por grupo, no próprio notebook, com o [Mosquitto](#).

Etapa 2: o `mosquitto.conf`

Linha	Por quê
<code>allow_anonymous false</code>	ninguém entra sem usuário e senha
<code>password_file broker/senhas</code>	onde estão os usuários
<code>acl_file broker/acl</code>	onde estão as permissões
<code>persistence false</code>	começa vazio a cada reinício
<code>listener 1883</code>	gateway e middleware
<code>listener 9001 + protocol websockets</code>	app no navegador do celular

Etapa 2: a acl

Usuário	Publica (write)	Assina (read)
<i>gateway</i>	telemetria, resposta, estado	os dois comandos
<i>middleware</i>	nada	tudo da bancada
<i>app_leitura</i>	nada	telemetria, resposta, estado
<i>operador</i>	os dois comandos	telemetria, resposta, estado

user gateway

topic write bancadaN/telemetria/#

O que não está escrito é **proibido**.

Etapa 2: usuários e checkpoint

```
mosquitto_passwd -c -b broker\senhas gateway SENHA
```

-c só no primeiro usuário: ele recria o arquivo.

```
mosquitto -c broker\mosquitto.conf -v
```

MQTT Explorer em 127.0.0.1:1883

Esperado

sem usuário

recusada

operador, senha errada

recusada

operador, senha certa

conecta

Etapa 3: conceitos do MQTT

Conceito No projeto

Tópico	<i>bancada3/telemetria/temperatura</i>
QoS 0	telemetria: perder uma amostra não importa
QoS 1	status e comandos: com confirmação
Retida	status: o app já abre sabendo o estado
Last Will	broker avisa <i>offline</i> se o gateway cair

Etapa 3: os tópicos da bancada

Tópico	Publica	QoS	Retido
<i>bancadaN/telemetria/temperatura</i>	gateway	0	não
<i>bancadaN/telemetria/velocidade</i>	gateway	0	não
<i>bancadaN/telemetria/status</i>	gateway	1	sim
<i>bancadaN/comando/velocidade</i>	operador	1	nunca
<i>bancadaN/comando/motor</i>	operador	1	nunca
<i>bancadaN/comando/resposta</i>	gateway	1	não
<i>bancadaN/gateway/estado</i>	gateway	1	sim

Etapa 3: o que escrever

conectar_broker

- `mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, ...)`
- usuário e senha, `will_set` antes de conectar
- callbacks, `connect_async`, `loop_start`

publicar e publicar_telemetria

```
{"valor": 23.5, "unidade": "°C", "ts": "2026-10-01T14:32:05Z"}
```

Etapa 3: checkpoint

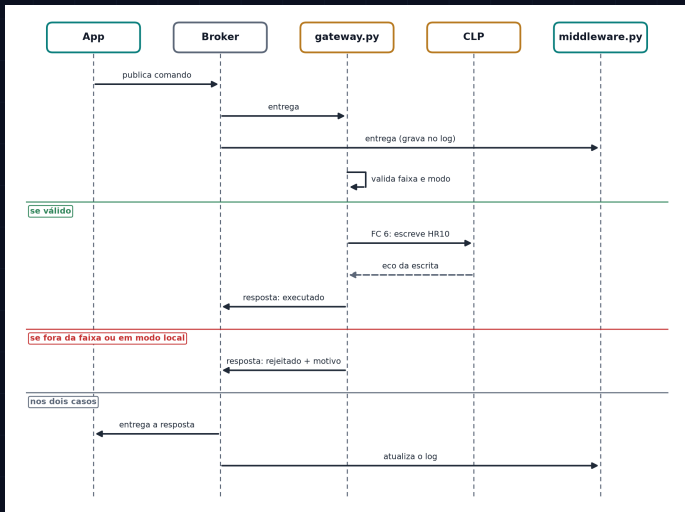
- MQTT Explorer como *operador*: a sua *bancadaN* aparece
- Na janela do broker: a conexão do *gateway* e cada publicação

Experimento: Last Will

Feche a janela do gateway, sem Ctrl+C.

Em alguns segundos, *gateway/estado* vira *offline*: quem publicou foi o **broker**.

Etapa 4: o caminho do comando



Etapa 4: por que uma fila

Duas threads

ao_receber: thread do MQTT, a qualquer momento.

Laço principal: lê o CLP a cada segundo.

Uma conexão Modbus

As duas no mesmo socket embaralham as requisições.

ao_receber só **enfileira**. Quem escreve é sempre o laço.

Etapa 4: validar antes de escrever

Passo	Confere	Se falhar
1	JSON com <i>id</i> e <i>valor</i> numérico	<i>formato invalido</i>
2	velocidade entre 0 e 1700	<i>fora da faixa</i>
2	motor em 0, 1 ou 2	<i>valor invalido</i>
3	bancada em modo remoto	<i>modo local</i>
4	escrita FC 6 respondeu	<i>falhou</i>

E o **heartbeat**: sem ele o CLP nunca deixa o motor ligar.

Etapa 4: checkpoint

MQTT Explorer, usuário *operador*:

Tópico	Mensagem	Resposta
<i>velocidade</i>	<code>{"id": "t1", "valor": 600}</code>	<i>executado</i>
<i>motor</i>	<code>{"id": "t2", "valor": 1}</code>	motor parte
<i>velocidade</i>	<code>{"id": "t3", "valor": 9000}</code>	<i>fora da faixa</i>
<i>motor</i>	<code>{"id": "t4", "valor": 7}</code>	<i>valor invalido</i>
<i>motor</i> (local)	<code>{"id": "t5", "valor": 1}</code>	<i>modo local</i>

Motor só liga com o monitor olhando.

Etapa 4: para pensar

Por que comando nunca é retido?

O broker guardaria o último *ligar*.

O gateway reinicia, assina o tópico, recebe o *ligar* antigo...
e o motor liga sozinho.

Por que a faixa é checada no gateway e no CLP?

Nenhuma camada confia na de cima (ADR-005).

Etapa 5: o middleware

Assina *bancadaN/#*, grava no SQLite, serve o app. **Nunca** fala com o CLP.

Mensagem	Grava com
<i>telemetria/<grandeza></i>	<i>gravar_medicao</i>
<i>comando/velocidade, motor</i>	<i>gravar_comando</i>
<i>comando/resposta</i>	<i>gravar_resposta</i>

Etapa 5: o histórico

```
SELECT ts, valor, unidade FROM medicoes
```

```
WHERE grandeza = ? AND ts >= ? ORDER BY ts
```

Os ? não são detalhe

Quem preenche é o SQLite. Montar SQL juntando texto com + ou f-string abre espaço para [injeção de SQL](#).

Etapa 5: tudo junto

```
.\iniciar.ps1
```

1. Notebook: `http://127.0.0.1:8000`
2. Celular no Wi-Fi do laboratório: o endereço que o script mostrou
3. Entrar como *operador* e comandar
4. Bônus: `http://127.0.0.1:8000/docs`

Windows perguntou sobre a rede? **Permitir**, rede privada.

Etapa 6: segurança em camadas

Camada	Proteção
--------	----------

Hardware	emergência corta a potência, sem software
----------	---

CLP	modo local, limite, watchdog, falha travada
-----	---

Gateway	valida antes de escrever; único na porta 502
---------	--

Broker	usuário por componente e ACL
--------	------------------------------

Cada camada cobre a falha da camada de cima.

6.1 Watchdog (o monitor faz)

Motor ligado pelo app. O monitor fecha a janela do gateway.

- Motor para em até 3 s
- App: *gateway desconectado* (Last Will)
- Status: *heartbeat perdido*, falha 2

Religou o gateway e o motor não voltou. Certo ou errado?

Certo: ninguém religa um motor sem reconhecer a falha.

6.2 e 6.3: modo local e ACL

Modo local

Chave em local, ligar pelo app:

rejeitado, modo local.

Ninguém liga remotamente um motor em que outra pessoa mexe.

ACL

Como *app_leitura*, publique *ligar* em *comando/motor*.

A **sua** ACL descarta. O motor não liga.

6.4 A senha trafega aberta

1. Wireshark: Adapter for loopback traffic capture
2. Filtro: `mqt`
3. Reinicie o gateway
4. Pacote Connect Command: User Name e Password

Decisão consciente (ADR-015)

TLS exigiria aceitar certificado em cada celular. Numa fábrica: porta 8883 com TLS.

Até onde este projeto protege

Alvo: SL 1 da IEC 62443 (ADR-029)

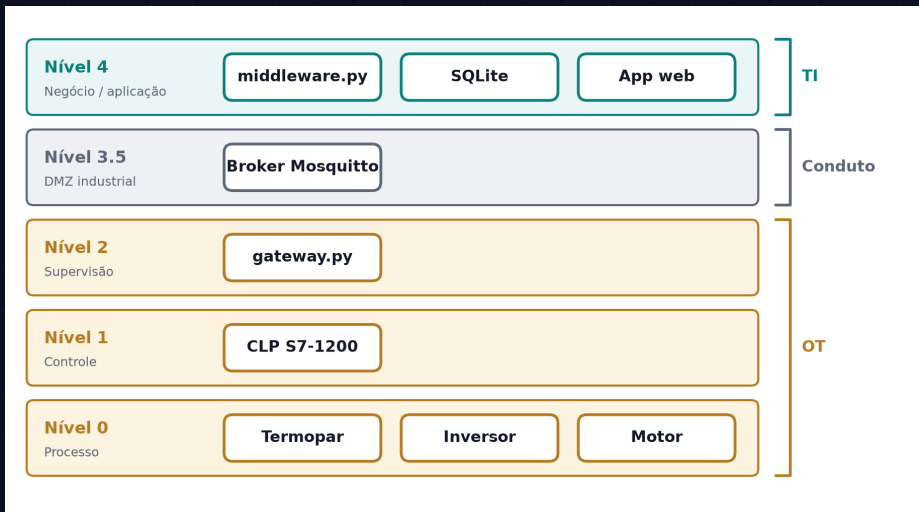
Protege contra erro e curiosidade, não contra ataque intencional.

Riscos aceitos:

- senha legível na rede da aula
- celulares na mesma sub-rede dos CLPs
- Modbus direto no CLP com o gateway desligado

O que seria preciso para chegar a SL 2?

Onde cada peça fica no Purdue



Entregável do grupo

1. *bancadaN_medicoes.csv*: link **Baixar medições** no app
2. Print do app no celular com o motor ligado
3. Respostas curtas às três perguntas abaixo

Perguntas

1. Em que nível Purdue fica cada peça que vocês construíram?
2. Qual o caminho de um comando, do toque ao motor, e em quais protocolos?
3. Cite um risco aceito e o que seria preciso para eliminá-lo.

Fechamento

O que vocês construíram

CLP → Modbus TCP → gateway → MQTT → broker → middleware e app → celular

e o caminho de volta, com validação em cada camada.

Todas as decisões, com o porquê de cada uma: *decisoões*/ (30 ADRs).